

ASSIGNMENT 3

BLOCK CHAIN

Ujjawal kumar Singh 22b1065

Nimish Manware 22b0944

Chandrakant Pradhan 22b1023

Implementation of .sol and .js files

1. Token.sol

The `Token.sol` contract implements an **ERC-20 token** with an initial supply of 1,000,000 tokens minted to the deployer's address. It inherits from OpenZeppelin's `ERC20` and `ERC20Permit` contracts

2. LPToken.sol

We implemented a separate `LPToken` to ensure accurate fee distribution among liquidity providers. When liquidity is **added** (`mint`), the LP's share and reward snapshot are stored. When liquidity is **removed** (`burn`), the earned rewards—based on their share of `totalLiquidity`—are transferred, and their data is cleared.

Swap fees accumulate in the reserves, and rewards are distributed only during mint or burn. This ensures fairness, as only active LPs at the time of swaps receive fees. Mappings track each LP's liquidity and rewards snapshot.

3. Dex.sol

The `Dex.sol` contract is a decentralized exchange that allows users to swap tokens, add liquidity, and remove liquidity using ERC-20 tokens and custom LP tokens functionality.

Constructor: Initializes the contract with two tokens (`tokenA` and `tokenB`) and creates an LP token contract with same owner as `Dex.sol`

Liquidity Functions:

`addLiquidity`: Adds liquidity to the pool and mints LP tokens adds to `reserveA` and `reserveB` by updating it

removeLiquidity: Allows liquidity providers to withdraw tokens and claim rewards. The claim rewards are collected as swap fees which gets distributed according to the ratio of liquidity of lp providers and provided at **claim rewards** functional call.

Swap Function: Allows users to swap between **tokenA** and **tokenB**, with a small fee taken and distributed to LP token holders.

Utility: Functions to get reserves and token prices in the pool.

The contract uses the **nonReentrant** modifier to prevent reentrancy attacks and ensures safe token transfers.

4. Arbitrage.sol

The **Arbitrage.sol** contract enables an arbitrage strategy by interacting with two decentralized exchanges (DEXes) to exploit price discrepancies between them for profit.

Constructor: Initializes the contract with two DEX addresses and two tokens (**tokenA** and **tokenB**). It ensures the provided addresses are valid and different from each other.

Arbitrage Execution:

The contract checks two arbitrage routes: from **tokenA** to **tokenB** and vice versa. It calculates potential profits for each route and compares them with a predefined minimum profit threshold (**MINIMUM_PROFIT_THRESHOLD**). If a profitable route is found, it executes the swaps on both DEXes, transferring tokens and capturing the profit.

Liquidity and Price Check:

Before performing any trades, it ensures that the liquidity is available on both DEXes. It uses a formula to calculate the amount of tokens to swap based on available reserves in the DEXes, factoring in fees.

Profit Calculation:

After performing the swaps, it checks if the final profit is greater than the minimum threshold.

If no profit is made, the contract skips the arbitrage opportunity and refunds the initial funds.

Events:

ArbitrageExecuted: Emitted when a successful arbitrage is executed with details of the trade and profit.

ArbitrageSkipped: Emitted when an arbitrage opportunity is skipped, with reasons such as insufficient profit, liquidity issues, or equal prices on both DEXes.

This contract leverages the price differences between two DEXes to perform arbitrage in an automated manner, ensuring that only profitable opportunities are executed, and managing the risks of insufficient liquidity and low profit.

5. Simulate_DEX.js

The `simulate_DEX.js` script models **realistic user activity** on a deployed `Dex.sol` contract, along with its associated `Token.sol` and `LPToken.sol` contracts. Its primary goal is to observe the DEX's dynamic behavior under simulated load.

Setup involves configuring simulation parameters like the number of transactions and participants (LPs, Traders). It connects to the deployed contracts using their ABIs and addresses via Web3.js, fetches blockchain accounts, and assigns roles. These accounts are then funded with initial Token A and Token B, about **10000 tokens** each and grant the DEX contract approval to manage their tokens. Crucially, the simulation bootstraps the market by having an initial LP add liquidity.

The **core simulation loop** runs for a predefined number of transactions. In each iteration, it randomly selects an action (**Swap, Add Liquidity, Remove Liquidity**) and an appropriate agent (Trader or LP). It calculates valid transaction parameters (like swap or liquidity amounts) based on agent balances, current DEX reserves, and random factors, ensuring realistic constraints. The chosen action is then executed by invoking the corresponding function (`swap`, `addLiquidity`, `removeLiquidity`) on the deployed `Dex.sol` contract instance.

Throughout the simulation, **key metrics** are collected after each transaction attempt and stored temporarily in data structures (like arrays). These metrics

include pool reserves (**Token A and B**), the calculated **spot price**, individual LP shares (via LP token balances), and **transaction slippage (for swaps)**. Upon completion, the script outputs this detailed **time-series data** and aggregate statistics to the terminal, facilitating analysis of price movement, liquidity dynamics, and slippage patterns.

6. Simulate_arbitrage.js

The `simulate_Arbitrage.js` script tests deployed **Arbitrage.sol** contract. It interacts between this contract and two distinct **DEX** instances (**DEX1**, **DEX2**), to verify the arbitrage contract's ability to detect and capitalize on price discrepancies between them for a given token pair (**Token A**, **Token B**).

Setup involves configuring contract addresses and connecting to them via Web3.js. Crucially, it establishes the necessary conditions for potential arbitrage by initializing DEX1 and DEX2 with different liquidity ratios using a dedicated LP account where `account[2]` was used, thereby creating an initial price difference. It also prepares the designated arbitrageur account, `account[1]` by funding it with 1000 tokens and ensuring it grants necessary token approvals to the **Arbitrage.sol** contract address, allowing the contract to execute swaps on its behalf based on conditions that are written in the `.sol` file.

The **Execution phase** calls its `executeArbitrage` function from the prepared arbitrageur account. The script explicitly runs through different scenarios – typically one designed with a clear price gap where arbitrage should be profitable, and another where prices are deliberately brought closer.

The script prints the log for `ArbitrageExecuted` and `ArbitrageSkipped` events emitted by the `Arbitrage.sol` contract to understand the decisions made. It tracks the token balances of the arbitrageur account before and after attempts to measure realized profit or loss. All findings, including event details, balance changes, and price movements, are logged to the console for analysis.

Security and Validations

There were no basic checks required in `Token.sol` and `LPToken.sol` other files had validation checks like in `dex.sol`

- The checks in .sol files are executed through require statements , these statements checks or assigns the statements and if there is any issue they just show error message which is there next input

Some common checks in DEX.sol

```
require(_tokenA != address(0) && _tokenB != address(0),
"DEX: Zero") require(_tokenA != _tokenB, "DEX: Identical
token addresses");

require(amountADesired > 0 && amountBDesired > 0, "Dex:
Insufficient amounts");    #during add liquidity

require(tokenA.transferFrom(msg.sender, address(this),
amountA), "Dex: TokenA transferFrom failed");    #ensuring
proper transfer take place

require(liquidityMinted > 0, "Dex: Insufficient liquidity
minted");

require(lpBalance >= liquidityToBurn, "Dex: Insufficient LP
balance");

require(amountInWithFee > 0, "Dex: Input amount too small
after fee");

require(reserveA > 0 && reserveB > 0, "Dex: NO_LIQUIDITY");
```

Checks in Arbitrage.sol

```
require(_dex1 != address(0) && _dex2 != address(0) &&
_tokenA != address(0) && _tokenB != address(0), "Arbitrage:
Zero address");

require(_dex1 != _dex2, "Arbitrage: DEX addresses same");

# some common constructor's checks

if (intermediateOut1 == 0 && intermediateOut2 == 0) {

    emit ArbitrageSkipped(tokenStartAddr,"Calculated swap
output is zero for both DEXes", 0);}

}
```

```

# unique type of check that is inter. amount becomes 0 in
swap then skip

require(finalBalance >= actualFinalAmountOut, "Arbitrage:
Final balance calculation error");

require(_tokenStartInstance.approve(address(dexBuyOn),
_amountIn), "Arbitrage: Approve dexBuyOn failed");
#setting of approval

require(_tokenStartInstance.approve(address(dexBuyOn), 0),
"Arbitrage: Reset1 failed"); #unsetting of approval

** all other checks were similar just different values so
not shown

```

Checks in simulate_DEX.sol

```

** during initialization

if (!web3.utils.isAddress(TOKEN_A_ADDRESS)) # existence of
that address

if (!tokenMetadata) #existence of metadata or token
address

** during simulation

if (lpBalanceA.isZero() || lpBalanceB.isZero()) {# continue
with metrical reports}

if (amountA_desired.isZero() || amountB_required.isZero())
{ #continue}

if (traderBalanceIn.isZero()) { #can't do swap}

const maxFromReserve = reserveIn.div(web3.utils.toBN(10));
const maxAmountIn = traderBalanceIn.lt(maxFromReserve) ?
traderBalanceIn : maxFromReserve; if
(maxAmountIn.isZero()) { #skipping}

```

```
catch (error) { console.error(`Transaction ${i + 1}
(${currentAction}) failed:`, error.message;} # if other
error comes in transaction execution
```

Checks in simulate_arbitrage.sol

```
** during initialization
```

```
if (!web3.utils.isAddress(TOKEN_A_ADDRESS)) # existence of
that address
```

```
if (!tokenMetadata) #existence of metadata or token
address
```

```
** setup liquidity
```

```
if (lpBalA.lt(web3.utils.toBN(liqA)) ||
lpBalB.lt(web3.utils.toBN(liqB))) {throw new Error(`LP
Account ${lpAcc} has insufficient balance to add required
liquidity.`);}
```

```
# if the lp balance A is less than liqA to be put on the
throw error
```

```
** simulate arbitrage
```

```
try { const txReceipt1 = await
arbitrage.methods.executeArbitrage(ARB_ATTEMPT_AMOUNT_A,
ARB_ATTEMPT_AMOUNT_B).send({ from: arbitrageurAccount, gas:
5000000 }); ....}
```

```
catch (er){ "arbitrage Tx1 failed"}
```

```
# this is important as some time txReceipt1 json can't read
the format of data
```

```
const executed = txReceipt2.events.ArbitrageExecuted;
if (executed) { # if this empty we shouldn't read this and
this can be empty}
```

Plots & Data Points

All the required Plots were generated through generate_table.py code in Testing folder with data in main_data.csv which has all the transactions information here the first liquidity of 5000 is provided lp[0] where each lp/trader account has 10000 each at start so initial lp is more of it and later on all will provide only 10% of its balance so there portion will be lesser . The generated graphs show that exactly.

Theoretical Questions

1. Only the DEX contract should be allowed to mint or burn LP tokens. This ensures that LP tokens are directly tied to the liquidity provided or removed through valid interactions with the DEX.

In the code the LPtoken.owner = DEx.onwer() so only it can call the functions mint and burn.

2. DEXs are permissionless and operate via smart contracts,

- it doesn't need any lower limit or upper limit to be an LP provider (except first minimum liquidity) .
- Every participant gets the same access to liquidity pools and pricing is determined by algorithms (like $x*y=k$), not by order books or market makers, reducing advantages from faster execution or special access.
- All trades are visible on the chain this will ensure transparency.

In the code , this is inherent in the constant product AMM model — no one can front-run within the smart contract once a transaction is mined, and all users interact via the same swap function

3. When users submit transactions to a DEX, they first enter the mempool — a public queue where miners pick which transactions to include in the next block. This visibility can be exploited by miners in several ways:

- If a miner notices a trade that is likely to yield a profit (such as an arbitrage opportunity), they can submit a similar transaction with a higher gas fee so their trade is processed first.

- After seeing a large trade that will significantly affect the token price, a miner may place their own trade immediately afterward to benefit from the resulting price movement.
- A miner might sandwich a user's transaction by submitting one trade before it (to shift the price) and one right after it (to sell at a more favorable rate), capturing profit on both sides at the expense of the original user.

Solutions to avoid this:

- Users can specify a maximum acceptable price change during a swap. If the price shifts beyond, the transaction is automatically reverted.
- Instead of submitting trade details directly, users first submit a hashed version of their transaction.
- By processing multiple transactions at once and using a uniform price for all, the DEX can make it harder for miners to exploit the order of transactions.

4.

- High gas fees can eat into or exceed arbitrage profits, making small profit opportunities non-viable.
- It can also disincentivize frequent LP actions (add/remove liquidity) and this will reduce liquidity hence bad prices for everyone.
- Small trades will become pointless. We will lose money if we initiate a small trade. This will result in only big trades making sense, pushing out small investors.
- Without arbitrage, there would be huge differences between multiple DEXs' arbitrage helps in maintaining price but since it is discouraged there will be huge gaps among DEXs

5. Yes. (gas fees are the same as transaction fees) ,Those who can pay higher gas fees can prioritize their transactions and front-run or sandwich others. This can be exploited by bots or well-funded miners.

So, again this will ensure only high gas transactions take place and small investors will be pushed at the end of the queue every time.

6.

- Use of large liquidity pools. If we ensure large pools indeed then whatever the swapping it will be some small fraction only till the point it is a smaller fraction of the whole pool it can't change the majority and slippage remains lesser or say the spotPrice remains constant.

- Another method could be to break large trades into smaller ones then also we can reduce slippage as again it is a small fraction of pool which won't change much.
- Fix a maximum cap on swap otherwise it will change the whole pool so there should be maximum cap on the swap as low fraction slippage is low.

7. In constant product AMM case, ($x \cdot y = k$)

$$x \cdot y = k$$

After deposit: $x' = x(1 + f)$, $y' = y(1 - g)$

$$x'y' = xy \Rightarrow (1 + f)(1 - g) = 1$$

$$1 - g = \frac{1}{1 + f}$$

$$g = \frac{f}{1 + f}$$

$$S = \frac{\text{Actual price}}{\text{Initial price}} - 1$$

Initial price $P_0 = \frac{y}{x}$

Actual price after swap:

$$P = \frac{\text{token } Y \text{ received}}{\text{token } X \text{ deposited}} = \frac{g \cdot y}{f \cdot x}$$

Substitute $g = \frac{f}{1+f}$:

$$P = \frac{\left(\frac{f}{1+f} \cdot y\right)}{f \cdot x} = \frac{y}{x(1+f)}$$

Now,

$$S = \frac{P}{P_0} - 1 = \frac{\frac{y}{x(1+f)}}{\frac{y}{x}} - 1 = \frac{1}{1+f} - 1$$

$$S = -\frac{f}{1+f}$$

Percentage of slippage vs trade lot fraction



